Date.prototype.getFullYear

📄 Título

Date.prototype.getFullYear() – El obtenedor del año completo

📝 Definición

`Date.prototype.getFullYear()` es un método de JavaScript que devuelve el **año completo** (de 4 dígitos) de un objeto `Date` según la **hora local**.

🧠 ¿Qué es?

Es el método estándar para obtener el año de una fecha en JavaScript.

A diferencia de `getYear()` (que está obsoleto y devuelve el año menos 1900), `getFullYear()` devuelve el año **completo de 4 dígitos** (ej: 2024 en lugar de 124).

El método es **local**: devuelve el año según la zona horaria del sistema donde se ejecuta el código. Para obtener el año en UTC, se usa `getUTCFullYear()`.

🎯 ¿Para qué sirve?

- Obtener el año actual para **mostrar en interfaces**.
- Calcular la **edad** de una persona.
- Generar **años fiscales** y **períodos contables**.
- Filtrar datos por **año específico**.
- Crear **calendarios y agendas**.
- Validar **rangos de fechas**.
- Generar **números de serie** con año.
- Calcular **diferencias entre años**.
- Implementar **lógica de negocio** basada en años (ej: renovaciones anuales).

⚙️ ¿Cómo funciona?

El método obtiene el año del objeto `Date` basado en la **hora local** del sistema.

Consideraciones importantes:

- Devuelve un número entero de 4 dígitos (2000-9999).
- Para años antes del 2000, devuelve el año completo (ej: 1999).
- El año 0 no existe en el calendario gregoriano (el año 1 a.C. es el año 1).
- Los años negativos representan años antes de Cristo.
- El método es **getter** (no modifica el objeto original).

📝 Sintaxis / Estructura (JavaScript)

```
date.getFullYear()
```

Retorna:

- Un número entero que representa el año (4 dígitos).

Errores:

- Si el objeto `Date` es inválido, devuelve `NaN`.

💡 Ejemplos

```
// 1. Uso básico
const date = new Date(2024, 11, 25);
console.log(date.getFullYear()); // 2024

// 2. Fecha actual
const now = new Date();
console.log(now.getFullYear()); // Año actual (ej: 2026)

// 3. Años antes del 2000
const oldDate = new Date(1985, 6, 15);
console.log(oldDate.getFullYear()); // 1985

// 4. Años en el futuro
```

```
const futureDate = new Date(2050, 0, 1);
console.log(futureDate.getFullYear()); // 2050

// 5. Años negativos (a.C.)
const ancientDate = new Date(-500, 0, 1);
console.log(ancientDate.getFullYear()); // -500

// 6. Año 2000 (bisiesto)
const y2k = new Date(2000, 0, 1);
console.log(y2k.getFullYear()); // 2000

// 7. Comparación con getYear() (obsoleto)
const dateObj = new Date(2024, 0, 1);
console.log(dateObj.getFullYear()); // 2024
console.log(dateObj.getYear()); // 124 (obsoleto, año - 190
0)

// 8. Verificar si un año es bisiesto
function isLeapYear(year) {
  return (year % 4 === 0 && year % 100 !== 0) || (year % 40
0 === 0);
}

console.log(isLeapYear(2024)); // true
console.log(isLeapYear(2023)); // false
console.log(isLeapYear(2000)); // true (año divisible por 4
00)
console.log(isLeapYear(1900)); // false (divisible por 100
pero no por 400)

// 9. Calcular edad
function calculateAge(birthDate) {
  const today = new Date();
  const birthYear = birthDate.getFullYear();
  const currentYear = today.getFullYear();

  let age = currentYear - birthYear;
```

```
  // Ajustar si aún no ha cumplido años este año
  const birthMonth = birthDate.getMonth();
  const currentMonth = today.getMonth();
  const birthDay = birthDate.getDate();
  const currentDay = today.getDate();

  if (currentMonth < birthMonth ||
      (currentMonth === birthMonth && currentDay < birthDa
y)) {
    age--;
  }

  return age;
}

console.log(calculateAge(new Date(1990, 5, 15))); // Edad a
proximada

// 10. Obtener año con formato de 4 dígitos siempre
function getFullYearString(date) {
  return date.getFullYear().toString().padStart(4, '0');
}

console.log(getFullYearString(new Date(2024, 0, 1))); // "2
024"
console.log(getFullYearString(new Date(99, 0, 1))); // "199
9"

// 11. Generar año fiscal (comienza en octubre)
function getFiscalYear(date) {
  const year = date.getFullYear();
  const month = date.getMonth(); // 0-11
  return month >= 9 ? year + 1 : year; // Si es octubre o d
espués, año fiscal siguiente
}

console.log(getFiscalYear(new Date(2024, 8, 1))); // 2024
(septiembre)
```

```
console.log(getFiscalYear(new Date(2024, 9, 1))); // 2025
(octubre)

// 12. Rango de años (década actual)
function getCurrentDecade() {
  const year = new Date().getFullYear();
  const start = Math.floor(year / 10) * 10;
  return `${start}-${start + 9}`;
}

console.log(getCurrentDecade()); // "2020-2029"

// 13. Obtener siglo
function getCentury(date) {
  const year = date.getFullYear();
  return Math.ceil(year / 100);
}

console.log(getCentury(new Date(2024, 0, 1))); // 21
console.log(getCentury(new Date(1999, 0, 1))); // 20

// 14. Verificar año en rango
function isInYearRange(date, startYear, endYear) {
  const year = date.getFullYear();
  return year >= startYear && year <= endYear;
}

console.log(isInYearRange(new Date(2024, 0, 1), 2020, 202
5)); // true
console.log(isInYearRange(new Date(2019, 0, 1), 2020, 202
5)); // false

// 15. Años con nombres (chino, japonés, etc.)
function getChineseZodiac(year) {
  const zodiacs = ['Rata', 'Buey', 'Tigre', 'Conejo', 'Drag
ón',
                   'Serpiente', 'Caballo', 'Cabra', 'Mono',
                   'Gallo', 'Perro', 'Cerdo'];
```

```
  return zodiacs[(year - 1900) % 12];
}

console.log(getChineseZodiac(2024)); // "Dragón"
console.log(getChineseZodiac(2025)); // "Serpiente"

// 16. Obtener año con zona horaria específica
function getYearInTimezone(date, timezone) {
  const options = { timeZone: timezone, year: 'numeric' };
  return parseInt(date.toLocaleDateString('en-US', option
s));
}

console.log(getYearInTimezone(new Date(), 'America/New_Yor
k'));
console.log(getYearInTimezone(new Date(), 'Asia/Tokyo'));

// 17. Diferencia en años entre dos fechas
function yearsBetween(date1, date2) {
  return Math.abs(date1.getFullYear() - date2.getFullYear
());
}

console.log(yearsBetween(new Date(2000, 0, 1), new Date(202
4, 0, 1))); // 24

// 18. Año en diferentes calendarios
function getYearInCalendar(date, calendar) {
  const options = { calendar: calendar, year: 'numeric' };
  return date.toLocaleDateString('en-US', options);
}

console.log(getYearInCalendar(new Date(2024, 0, 1), 'chines
e')); // Año chino
console.log(getYearInCalendar(new Date(2024, 0, 1), 'islami
c')); // Año islámico
console.log(getYearInCalendar(new Date(2024, 0, 1), 'buddhi
st')); // Año budista
```

```
// 19. Validación de año
function isValidYear(year) {
  return Number.isInteger(year) && year >= 0 && year <= 999
9;
}

console.log(isValidYear(2024)); // true
console.log(isValidYear(-1)); // false
console.log(isValidYear(10000)); // false

// 20. Años para generación
function getGeneration(year) {
  if (year >= 1946 && year <= 1964) return 'Baby Boomer';
  if (year >= 1965 && year <= 1980) return 'Generación X';
  if (year >= 1981 && year <= 1996) return 'Millennial (Gen
Y)';
  if (year >= 1997 && year <= 2012) return 'Generación Z';
  if (year >= 2013) return 'Generación Alpha';
  return 'Generación desconocida';
}

console.log(getGeneration(1990)); // "Millennial (Gen Y)"
console.log(getGeneration(2005)); // "Generación Z"
```

⚠️ Errores comunes

- **Usar `getYear()` en lugar de `getFullYear()`**:

```
// ❌ MAL (obsoleto)
const year = date.getYear(); // 124 (para 2024)

// ✅ BIEN
const year = date.getFullYear(); // 2024
```

- **Asumir que los años negativos son válidos**: JavaScript los soporta, pero pueden causar confusión.

- **No considerar la zona horaria**: `getFullYear()` es local, no UTC.
- **Olvidar que enero es 0**: No afecta a `getFullYear()`.
- **Pensar que `getFullYear()` devuelve un string**: Devuelve un número.

💼 Casos de uso

1. **Sistema de autenticación con expiración anual**:

```
class LicenseManager {
  constructor() {
    this.licenses = new Map();
  }

  createLicense(userId) {
    const now = new Date();
    const expiryYear = now.getFullYear() + 1;
    const expiryDate = new Date(expiryYear, 11, 31); //
31 de diciembre del año siguiente

    this.licenses.set(userId, {
      created: now,
      expiry: expiryDate,
      expiryYear: expiryYear,
      active: true
    });

    return { userId, expiryYear };
  }

  isLicenseValid(userId) {
    const license = this.licenses.get(userId);
    if (!license) return false;
    if (!license.active) return false;

    const now = new Date();
    const currentYear = now.getFullYear();
```

```
    // Si el año actual es mayor que el año de expiració
n, está expirado
    if (currentYear > license.expiryYear) return false;

    // Si es el mismo año, verificar si ya pasó la fecha
    if (currentYear === license.expiryYear) {
      return now <= license.expiry;
    }

    return true;
  }
}
```

2. **Generador de números de factura**:

```
class InvoiceGenerator {
  constructor() {
    this.counter = 0;
  }

  generateInvoice(prefix = 'INV') {
    const now = new Date();
    const year = now.getFullYear();
    this.counter++;

    return `${prefix}-${year}-${this.counter.toString().
padStart(6, '0')}`;
  }

  getYearlyReport(year) {
    // Simula obtener reporte del año
    return {
      year,
      totalInvoices: Math.floor(Math.random() * 1000),
      totalAmount: Math.floor(Math.random() * 1000000),
      generation: `Año fiscal${year}`
    };
```

```
  }
}

const generator = new InvoiceGenerator();
console.log(generator.generateInvoice('FAC')); // "FAC-2
024-000001"
```

3. **Filtro de eventos por año**:

```
class EventFilter {
  constructor(events) {
    this.events = events.map(e => ({
      ...e,
      date: new Date(e.date)
    }));
  }

  filterByYear(year) {
    return this.events
      .filter(e => e.date.getFullYear() === year)
      .sort((a, b) => a.date - b.date);
  }

  getYearSummary(year) {
    const events = this.filterByYear(year);
    const months = {};

    events.forEach(e => {
      const month = e.date.getMonth();
      months[month] = (months[month] || 0) + 1;
    });

    return {
      year,
      totalEvents: events.length,
      events,
      months: months,
```

```
      monthNames: Object.keys(months).map(m =>
        new Date(year, parseInt(m), 1).toLocaleDateStrin
g('es-ES', { month: 'long' })
      )
    };
  }
}

const events = [
  { id: 1, name: 'Evento 1', date: '2024-06-15' },
  { id: 2, name: 'Evento 2', date: '2024-12-25' },
  { id: 3, name: 'Evento 3', date: '2023-12-31' }
];

const filter = new EventFilter(events);
console.log(filter.filterByYear(2024));
```

4. **Calculadora de edad detallada**:

```
class AgeCalculator {
  constructor(birthDate) {
    this.birthDate = new Date(birthDate);
  }

  getAgeOn(targetDate = new Date()) {
    const target = new Date(targetDate);
    const birthYear = this.birthDate.getFullYear();
    const targetYear = target.getFullYear();

    let age = targetYear - birthYear;

    const birthMonth = this.birthDate.getMonth();
    const targetMonth = target.getMonth();
    const birthDay = this.birthDate.getDate();
    const targetDay = target.getDate();

    if (targetMonth < birthMonth ||
        (targetMonth === birthMonth && targetDay < birth
```

```
Day)) {
      age--;
    }

    return age;
  }

  getAgeInYearsMonthsDays(targetDate = new Date()) {
    const target = new Date(targetDate);
    let years = this.getAgeOn(target);

    const anniversary = new Date(this.birthDate);
    anniversary.setFullYear(anniversary.getFullYear() +
years);

    let months = target.getMonth() - anniversary.getMont
h();
    if (months < 0) {
      months += 12;
      years--;
    }

    let days = target.getDate() - anniversary.getDate();
    if (days < 0) {
      const prevMonth = new Date(target.getFullYear(), t
arget.getMonth(), 0);
      days += prevMonth.getDate();
      months--;
    }

    return { years, months, days };
  }

  getZodiacSign() {
    const month = this.birthDate.getMonth() + 1;
    const day = this.birthDate.getDate();

    const signs = [
```

```
      { start: [1, 20], end: [2, 18], name: 'Acuario' },
      { start: [2, 19], end: [3, 20], name: 'Piscis' },
      { start: [3, 21], end: [4, 19], name: 'Aries' },
      { start: [4, 20], end: [5, 20], name: 'Tauro' },
      { start: [5, 21], end: [6, 20], name: 'Géminis' },
      { start: [6, 21], end: [7, 22], name: 'Cáncer' },
      { start: [7, 23], end: [8, 22], name: 'Leo' },
      { start: [8, 23], end: [9, 22], name: 'Virgo' },
      { start: [9, 23], end: [10, 22], name: 'Libra' },
      { start: [10, 23], end: [11, 21], name: 'Escorpio'
},
      { start: [11, 22], end: [12, 21], name: 'Sagitari
o' },
      { start: [12, 22], end: [1, 19], name: 'Capricorni
o' }
    ];

    for (const sign of signs) {
      const startMonth = sign.start[0];
      const startDay = sign.start[1];
      const endMonth = sign.end[0];
      const endDay = sign.end[1];

      if ((month === startMonth && day >= startDay) ||
          (month === endMonth && day <= endDay) ||
          (month > startMonth && month < endMonth) ||
          (startMonth > endMonth && (month >= startMonth
|| month <= endMonth))) {
        return sign.name;
      }
    }

    return 'Capricornio'; // Caso default
  }
}

const ageCalc = new AgeCalculator('1990-06-15');
console.log(ageCalc.getAgeOn(new Date(2024, 11, 25)));
```

```
console.log(ageCalc.getAgeInYearsMonthsDays());
console.log(ageCalc.getZodiacSign());
```

5. **Sistema de renovación de suscripción**:

```
class SubscriptionManager {
  constructor() {
    this.subscriptions = [];
  }

  createSubscription(userId, plan, startDate = new Date
()) {
    const start = new Date(startDate);
    const duration = plan === 'annual' ? 1 : 0;
    const end = new Date(start);
    end.setFullYear(end.getFullYear() + duration + 1);

    const subscription = {
      userId,
      plan,
      start,
      end,
      startYear: start.getFullYear(),
      endYear: end.getFullYear(),
      active: true,
      renewalYear: end.getFullYear()
    };

    this.subscriptions.push(subscription);
    return subscription;
  }

  getSubscriptionsForYear(year) {
    return this.subscriptions
      .filter(s => s.startYear <= year && s.endYear >= y
ear)
      .filter(s => s.active);
```

```
  }

  getRenewalsForYear(year) {
    return this.subscriptions
      .filter(s => s.renewalYear === year)
      .filter(s => s.active);
  }

  generateYearlyReport(year) {
    const active = this.getSubscriptionsForYear(year);
    const renewals = this.getRenewalsForYear(year);

    return {
      year,
      totalActive: active.length,
      totalRenewals: renewals.length,
      revenue: active.reduce((sum, s) => {
        return sum + (s.plan === 'annual' ? 120 : 12);
      }, 0)
    };
  }
}
```

6. **Generador de calendarios anuales**:

```
class YearCalendar {
  constructor(year = new Date().getFullYear()) {
    this.year = year;
    this.months = Array.from({ length: 12 }, (_, i) =>
({
      index: i,
      name: new Date(year, i, 1).toLocaleDateString('es-
ES', { month: 'long' }),
      days: this.getDaysInMonth(i)
    }));
  }
```

```
  getDaysInMonth(month) {
    const days = [];
    const daysInMonth = new Date(this.year, month + 1,
0).getDate();

    for (let day = 1; day <= daysInMonth; day++) {
      const date = new Date(this.year, month, day);
      days.push({
        day,
        weekday: date.getDay(),
        weekdayName: date.toLocaleDateString('es-ES', {
weekday: 'long' }),
        isWeekend: date.getDay() === 0 || date.getDay()
=== 6
      });
    }

    return days;
  }

  getStats() {
    const totalDays = 365 + (this.isLeapYear() ? 1 : 0);
    const firstDay = new Date(this.year, 0, 1).getDay();
    const lastDay = new Date(this.year, 11, 31).getDay
();

    return {
      year: this.year,
      totalDays,
      isLeapYear: this.isLeapYear(),
      firstDayOfYear: firstDay,
      lastDayOfYear: lastDay,
      weeks: Math.ceil((totalDays + firstDay) / 7)
    };
  }

  isLeapYear() {
    return (this.year % 4 === 0 && this.year % 100 !==
```

```
0) || (this.year % 400 === 0);
  }

  getHolidays() {
    // Ejemplo de días festivos fijos
    const holidays = [
      { date: new Date(this.year, 0, 1), name: 'Año Nuev
o' },
      { date: new Date(this.year, 0, 6), name: 'Día de R
eyes' },
      { date: new Date(this.year, 4, 1), name: 'Día del
Trabajo' },
      { date: new Date(this.year, 11, 25), name: 'Navida
d' }
    ];

    // Días festivos variables (ej: Semana Santa)
    // ... (cálculos complejos omitidos)

    return holidays;
  }
}

const calendar = new YearCalendar(2024);
console.log(calendar.getStats());
```

7. **Sistema de facturación con años fiscales**:

```
class FiscalYearManager {
  constructor(yearStart = 1) { // 1 = enero, 7 = julio,
etc.
    this.monthStart = yearStart;
  }

  getFiscalYear(date = new Date()) {
    const year = date.getFullYear();
    const month = date.getMonth() + 1;
```

```
    if (month >= this.monthStart) {
      return year;
    } else {
      return year - 1;
    }
  }

  getFiscalYearStart(fiscalYear) {
    return new Date(fiscalYear, this.monthStart - 1, 1);
  }

  getFiscalYearEnd(fiscalYear) {
    const end = new Date(fiscalYear + 1, this.monthStart
- 1, 0);
    return end;
  }

  getFiscalPeriod(date) {
    const fiscalYear = this.getFiscalYear(date);
    const start = this.getFiscalYearStart(fiscalYear);
    const end = this.getFiscalYearEnd(fiscalYear);
    const current = new Date(date);

    const totalDays = Math.floor((end - start) / (1000 *
60 * 60 * 24));
    const elapsedDays = Math.floor((current - start) /
(1000 * 60 * 60 * 24));

    return {
      fiscalYear,
      start,
      end,
      progress: Math.round((elapsedDays / totalDays) * 1
00),
      daysElapsed: elapsedDays,
      daysRemaining: totalDays - elapsedDays
    };
```

```
  }

  generateQuarterlyReport(year) {
    const quarters = [];
    for (let q = 0; q < 4; q++) {
      const startMonth = this.monthStart + q * 3;
      const start = new Date(year, startMonth - 1, 1);
      const end = new Date(year, startMonth + 2, 0);

      quarters.push({
        quarter: q + 1,
        start,
        end,
        label: `Q${q + 1}${year}`,
        months: Array.from({ length: 3 }, (_, i) =>
          new Date(year, startMonth - 1 + i, 1)
        )
      });
    }
    return quarters;
  }
}

const fiscal = new FiscalYearManager(1); // Año fiscal c
omienza en enero
console.log(fiscal.getFiscalPeriod(new Date(2024, 6, 1
5)));
```

8. **Validación de años en formularios**:

```
class YearValidator {
  constructor(minYear = 1900, maxYear = new Date().getFu
llYear()) {
    this.minYear = minYear;
    this.maxYear = maxYear;
  }
```

```
  validate(year) {
    const yearNum = parseInt(year);
    if (isNaN(yearNum)) {
      return { valid: false, message: 'Debe ser un núme
ro' };
    }

    if (yearNum < this.minYear) {
      return { valid: false, message: `El año debe ser m
ayor o igual a${this.minYear}` };
    }

    if (yearNum > this.maxYear) {
      return { valid: false, message: `El año debe ser m
enor o igual a${this.maxYear}` };
    }

    return { valid: true, year: yearNum };
  }

  getYearsInRange() {
    const years = [];
    for (let year = this.minYear; year <= this.maxYear;
year++) {
      years.push(year);
    }
    return years;
  }

  getYearOptions() {
    return this.getYearsInRange().map(year => ({
      value: year,
      label: year.toString(),
      selected: year === this.maxYear
    }));
  }
}
```

```
// Uso en un select de años
const validator = new YearValidator(2000, 2024);
console.log(validator.validate(2023));
console.log(validator.validate(1999));
```

9. **Predicción de años bisiestos**:

```
function getLeapYears(startYear, endYear) {
  const leapYears = [];
  for (let year = startYear; year <= endYear; year++) {
    if (isLeapYear(year)) {
      leapYears.push(year);
    }
  }
  return leapYears;
}

function getNextLeapYear(afterYear) {
  let year = afterYear + 1;
  while (!isLeapYear(year)) {
    year++;
  }
  return year;
}

function getLeapYearStats() {
  const now = new Date();
  const currentYear = now.getFullYear();
  const nextLeap = getNextLeapYear(currentYear);

  return {
    currentYear,
    isLeapYear: isLeapYear(currentYear),
    nextLeapYear: nextLeap,
    yearsUntilNextLeap: nextLeap - currentYear,
    leapYearsInDecade: getLeapYears(
      Math.floor(currentYear / 10) * 10,
```

```
      Math.floor(currentYear / 10) * 10 + 9
    )
  };
}

function isLeapYear(year) {
  return (year % 4 === 0 && year % 100 !== 0) || (year %
400 === 0);
}

console.log(getLeapYearStats());
```

10. **Análisis de tendencias anuales**:

```
class TrendAnalyzer {
  constructor(data) {
    this.data = data.map(d => ({
      ...d,
      year: new Date(d.date).getFullYear()
    }));
  }

  getYearlyTrends() {
    const trends = {};
    this.data.forEach(d => {
      if (!trends[d.year]) {
        trends[d.year] = { count: 0, total: 0, items:
[] };
      }
      trends[d.year].count++;
      trends[d.year].total += d.value || 0;
      trends[d.year].items.push(d);
    });

    // Ordenar años
    const sortedYears = Object.keys(trends).sort();
    const result = sortedYears.map(year => ({
```

```
      year: parseInt(year),
      count: trends[year].count,
      total: trends[year].total,
      average: trends[year].total / trends[year].count,
      growth: this.calculateGrowth(sortedYears, year, t
rends)
    }));

    return result;
  }

  calculateGrowth(sortedYears, currentYear, trends) {
    const index = sortedYears.indexOf(currentYear);
    if (index === 0) return 0;

    const prevYear = sortedYears[index - 1];
    const currentTotal = trends[currentYear].total;
    const prevTotal = trends[prevYear].total;

    if (prevTotal === 0) return currentTotal > 0 ? 100
: 0;
    return ((currentTotal - prevTotal) / prevTotal) * 1
00;
  }

  getYearWithMaxGrowth() {
    const trends = this.getYearlyTrends();
    return trends.reduce((max, trend) =>
      trend.growth > max.growth ? trend : max
    , trends[0] || { year: null, growth: 0 });
  }
}

const data = [
  { date: '2020-01-15', value: 100 },
  { date: '2020-06-20', value: 150 },
  { date: '2021-03-10', value: 200 },
  { date: '2021-09-05', value: 250 },
```

```
  { date: '2022-02-14', value: 300 }
];

const analyzer = new TrendAnalyzer(data);
console.log(analyzer.getYearlyTrends());
```

🧩 Relación con otros conceptos

- `Date.prototype.getUTCFullYear()`: Año en UTC.
- `Date.prototype.setFullYear()`: Establecer año.
- `Date.prototype.getYear()`: Obsoleto (año - 1900).
- `Date.prototype.getMonth()`: Mes (0-11).
- `Date.prototype.getDate()`: Día del mes.
- `Date.prototype.getDay()`: Día de la semana (0-6).
- `Date.prototype.getHours()`: Hora.
- `Date.prototype.getMilliseconds()`: Milisegundos.
- `Date.prototype.getTimezoneOffset()`: Diferencia horaria.

🧠 Analogía

Imagina que `getFullYear()` es como **leer el año en un calendario**. Miras la fecha y ves claramente "2024". Es simple, directo y siempre te da el número completo de 4 dígitos, sin abreviaturas ni confusiones.

📚 Fuente

- MDN Web Docs – Date.prototype.getFullYear()
- ECMAScript Specification – Date.prototype.getFullYear
- MDN – Date